Easy Maze

Riccardo Canonico (Mat. 335359)

Lorenzo Mastrini (Mat. 336478)

Indice dei contenuti

1. Descrizione del problema

2. Descrizione formale del dominio e dei vincoli:
    2.1 Rappresentazione formale del problema
    2.2 Vincoli dell'agente
    2.3 Vincoli del sistema

3. Machine Learning:
    3.1 Descrizione del modello
    3.2 Analisi prestazioni

4. Algoritmi in fase di risoluzione:
    4.1 Prestazioni degli algoritmi
    4.2 Confronti fra algoritmi

1. Descrizione del problema

1.1 Descrizione del problema

Il labirinto prevede una sola entrata ma può avere anche più uscite. È rappresentato come una matrice quadrata di celle in cui alcune sono calpestabili, mentre altre rappresentano i muri invalicabili. Le celle calpestabili hanno un costo associato, che rappresenta appunto il costo dell'azione per passare sulla cella.

Partendo da una posizione iniziale in una cella del bordo del labirinto, si vuole trovare la sequenza di azioni dell'agente per uscire.

Costo delle celle, muri, posizione iniziale e uscite sono codificati attraverso una lettera o un numero secondo la seguente legenda:

- S (Start): posizione iniziale dell'agente
- E (Exit): uscita
- X: muro invalicabile
- 0/4: cella calpestabile e costo per attraversarla

La struttura del labirinto da attraversare e la posizione della/e uscita/e sono passati al sistema tramite un'immagine.

2. Descrizione formale del dominio e dei vincoli

2.1 Rappresentazione formale del problema

- Stato iniziale (initial): coordinate (i, j) nella matrice della cella S (Start)
- Azioni (actions): [UP, DOWN, LEFT, RIGHT], rimuovendo eventualmente le azioni non effettuabili (se ci sono muri o l'agente si trova ai bordi della matrice)
- Risultati (result): dato lo stato attuale (coordinate dell'agente) e l'azione effettuata ritorna il nuovo stato (coordinate della cella in cui mi sono spostato)
- Goal Test (goal_test): torna True se lo stato attuale corrisponde alle coordinate di una delle uscite del labirinto (celle "E")
- Path Cost (path_cost): costo dallo stato iniziale S a un'uscita E, quindi per tutto il percorso
- Euristica (h): stima del costo del percorso. Tramite l'euristica di Manhattan calcoliamo il percorso più corto (minimo numero di celle attraversate) senza considerare le celle invalicabili, e poi seguiamo un approccio probabilistico: a priori calcoliamo le frequenze relative delle celle con un costo (da 0 a 4) e quelle con le 'X' che facciamo corrispondere a celle di costo 0, quindi per un totale di 5 diversi elementi (0/'X', 1, 2, 3, 4); a questo punto, in base al numero di celle ottenute con l'euristica di Manhattan calcoliamo una stima (tramite le frequenze trovate) di quante di queste celle hanno un certo costo, per poi sommare insieme questi valori, ognuno moltiplicato con il corrispondente costo:

  h = stima_celle__costo_0_nel_percorso_minimo * 0 +
  stima_celle__costo_1_nel_percorso_minimo * 1 +
  stima_celle__costo_2_nel_percorso_minimo * 2 + …

2.2 Vincoli dell'agente

v1 = l'agente può compiere un solo passo alla volta

v2 = l'agente si può muovere solo fra stanze adiacenti

v3 = l'agente non può muoversi fuori dal labirinto, cioè quando si trova lungo i bordi della matrice

v4 = l'agente può muoversi solo in stanze senza ostacoli

v5 = l’agente ad ogni passo può muoversi al massimo in 4 direzioni possibili: UP, DOWN, LEFT, RIGHT; nel caso in cui ci sia un ostacolo o il labirinto finisce si rimuovono le azioni corrispondenti alle mosse non disponibili

2.3 Vincoli del sistema

- La matrice e i numeri/lettere scritti a mano devono essere disegnati così da essere ben visibili dalla foto (per esempio tracciando linee abbastanza spesse)

- L’immagine passata al sistema dovrebbe contenere esclusivamente la matrice disegnata su un foglio bianco più pulito possibile

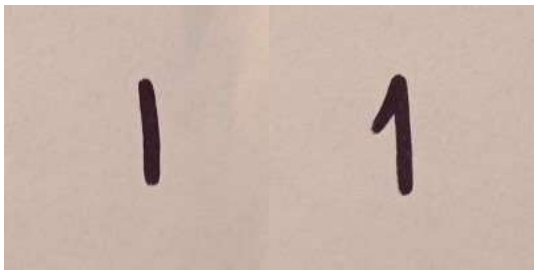
- Le linee che rappresentano la struttura della matrice dovrebbero essere più dritte possibili (orizzontalmente e verticalmente)

- Le linee costituenti la struttura della matrice devono essere ben collegate fra loro, senza lasciare spazi vuoti

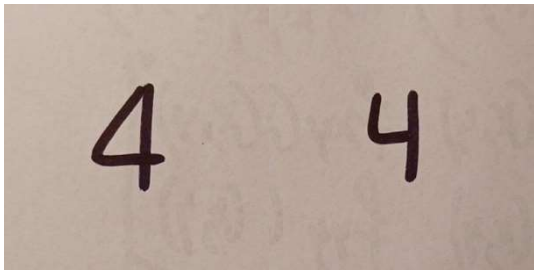
- I numeri/lettere all’interno delle singole celle dovrebbero essere sufficientemente distanziati/e dai bordi della cella stessa

- Per aiutare la rete neurale nel processo di classificazione ecco alcuni suggerimenti nella scrittura dei valori:

- I valori ‘1’ dovrebbero essere scritti senza la stanghetta laterale, o abbastanza corta:



- I valori ‘4’ dovrebbero essere scritti in questo modo, poiché nel dataset emnist sono rappresentati solo in questi formati:



3. Machine Learning

3.1 Descrizione del modello

I dati vengono passati al programma come una matrice scritta a mano che rappresenta il labirinto.

Per classificare le lettere/cifre all'interno della matrice è stata implementata una rete neurale tramite le librerie Keras e Scikit-learn.

Il dataset usato per l'addestramento della rete è emnist opportunamente modificato tenendo solo le lettere e cifre che ci interessano. Inoltre, sono state fatte delle operazioni aggiuntive di preprocessing sulle immagini del dataset, esattamente come vengono effettuate sulle immagini da classificare.

La rete neurale è strutturata in 4 layer:

- Initial Layer: formato dalle 784 feature di ogni immagine (28x28 pixel)
- Hidden Layer 1: formato da 512 nodi, ciascuno con funzione di attivazione ReLU
- Hidden Layer 2: formato da 256 nodi, ciascuno con funzione di attivazione ReLU
- Output Layer: formato da otto nodi con funzione d'attivazione Softmax, ciascuno corrispondente a una delle classi possibili

Alcuni iperparametri:

Learn Rate	0.001
Batch Size	64
Epoche	10
Funzione di ottimizzazione	Adam
Funzione di loss	Sparse Categorical Cross-Entropy
Metriche	Accuracy

Gli iperparametri e la struttura della rete sono stati scelti provando una serie di combinazioni tra diversi valori, fino ad arrivare ad un modello con buone prestazioni.

```python
def get_mlp_model(hiddenLayerOne=512, hiddenLayerTwo=256,
    dropout=0.5, learnRate=0.001):
    # initialize a sequential model and add layer to flatten the input data
    model = Sequential()

    model.add(Flatten(input_shape=(28, 28)))
    model.add(Dropout(dropout))
    # add two hidden layer with activation function relu and a dropout
    model.add(Dense(hiddenLayerOne, activation="relu"))
    model.add(Dropout(0.2))
    model.add(Dense(hiddenLayerTwo, activation="relu"))
    model.add(Dropout(0.2))

    # add a softmax layer on top
    model.add(Dense(num_classes, activation="softmax"))

    # compile the model
    model.compile(
        optimizer=Adam(learning_rate=learnRate),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"])

    # return compiled model
    return model
```

3.2 Analisi prestazioni

3.2.1 Fase di Training:

```
# stop earlier the training if it doesn't improve (the loss doesn't decrease)
keras_callbacks = [EarlyStopping(monitor='val_loss', patience=5, verbose=1, min_delta=0.001)]

# initialize model
print("[INFO] initializing model...")
model = get_mlp_model()

# train the network
print("[INFO] training model...")
H = model.fit(x=trainData, y=trainLabels, validation_split=0.15, batch_size=64, epochs=10, callbacks=keras_callbacks)
```

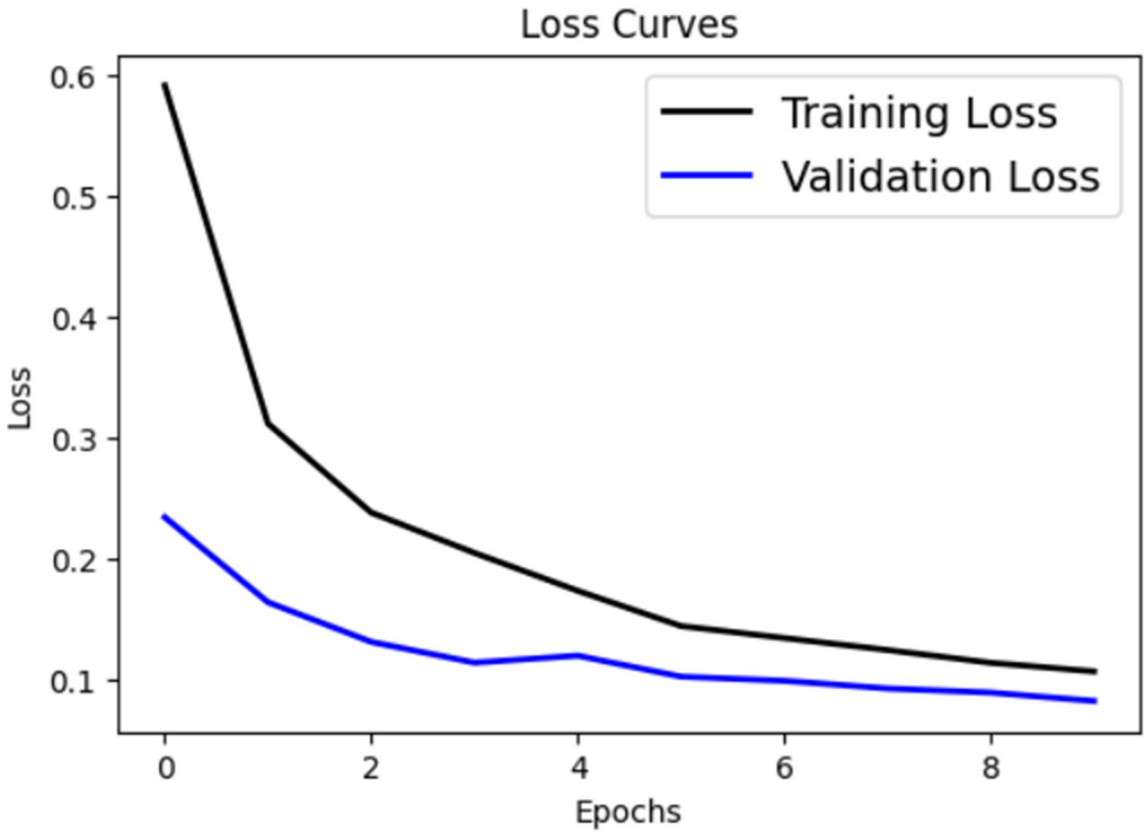
```
[INFO] initializing model...
[INFO] training model...
Epoch 1/10
255/255 [==============================] - 4s 13ms/step - loss: 0.5921 - accuracy: 0.8064 - val_loss: 0.2349 - val_accuracy: 0.9201
Epoch 2/10
255/255 [==============================] - 3s 13ms/step - loss: 0.3123 - accuracy: 0.9001 - val_loss: 0.1645 - val_accuracy: 0.9486
Epoch 3/10
255/255 [==============================] - 3s 11ms/step - loss: 0.2387 - accuracy: 0.9227 - val_loss: 0.1317 - val_accuracy: 0.9569
Epoch 4/10
255/255 [==============================] - 4s 17ms/step - loss: 0.2053 - accuracy: 0.9331 - val_loss: 0.1145 - val_accuracy: 0.9635
Epoch 5/10
255/255 [==============================] - 3s 12ms/step - loss: 0.1740 - accuracy: 0.9449 - val_loss: 0.1203 - val_accuracy: 0.9639
Epoch 6/10
255/255 [==============================] - 3s 11ms/step - loss: 0.1448 - accuracy: 0.9515 - val_loss: 0.1029 - val_accuracy: 0.9670
Epoch 7/10
255/255 [==============================] - 4s 14ms/step - loss: 0.1349 - accuracy: 0.9562 - val_loss: 0.0995 - val_accuracy: 0.9677
Epoch 8/10
255/255 [==============================] - 4s 15ms/step - loss: 0.1250 - accuracy: 0.9578 - val_loss: 0.0932 - val_accuracy: 0.9681
Epoch 9/10
255/255 [==============================] - 3s 11ms/step - loss: 0.1144 - accuracy: 0.9619 - val_loss: 0.0897 - val_accuracy: 0.9719
Epoch 10/10
255/255 [==============================] - 3s 12ms/step - loss: 0.1072 - accuracy: 0.9645 - val_loss: 0.0828 - val_accuracy: 0.9729
```

3.2.2 Accuracy sul test set:

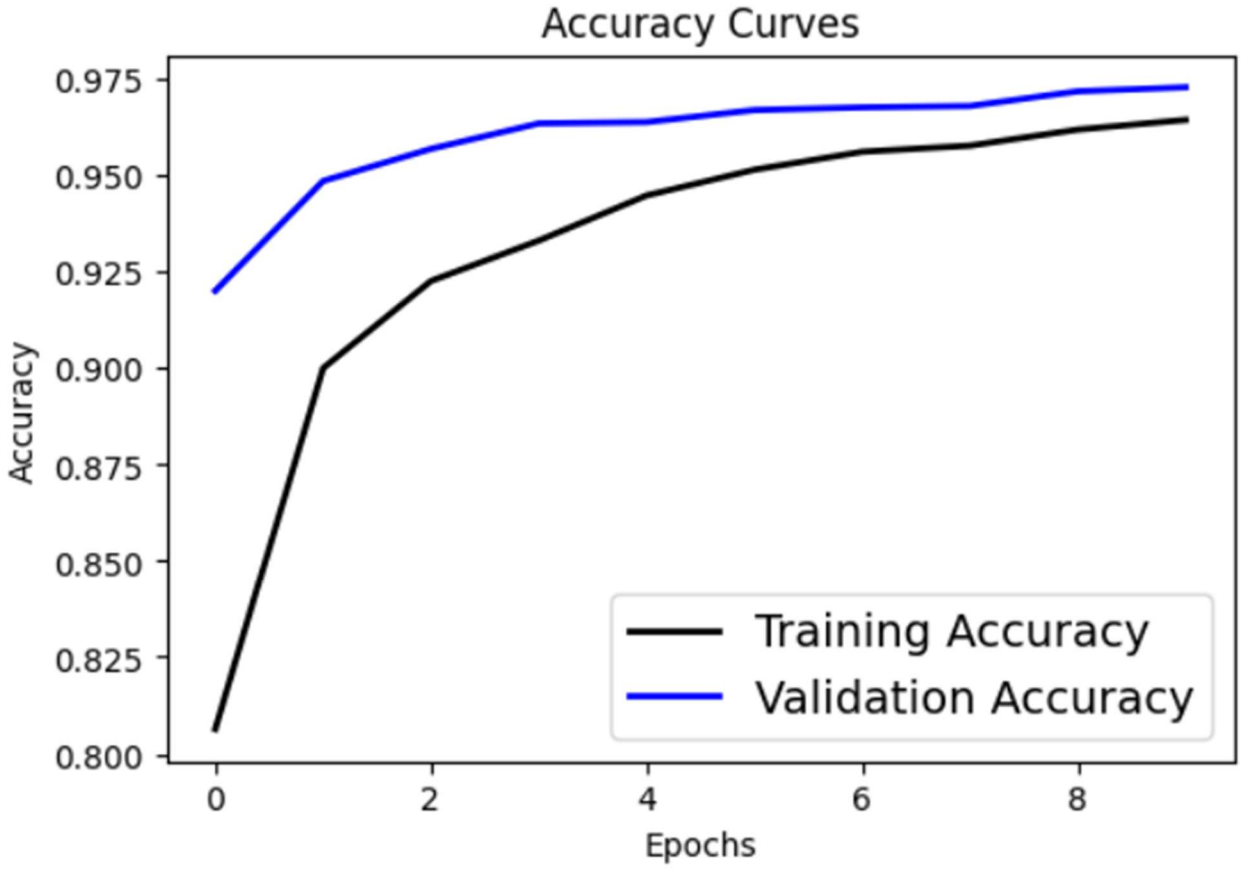
```
# make predictions on the test set and evaluate it
print("[INFO] evaluating network...")
accuracy = model.evaluate(testData, testLabels)[1]
print("accuracy: {:.2f}%".format(accuracy * 100)) # accuracy percentage
```

```
[INFO] evaluating network...
100/100 [==============================] - 1s 9ms/step - loss: 0.1097 - accuracy: 0.9688
accuracy: 96.88%
```

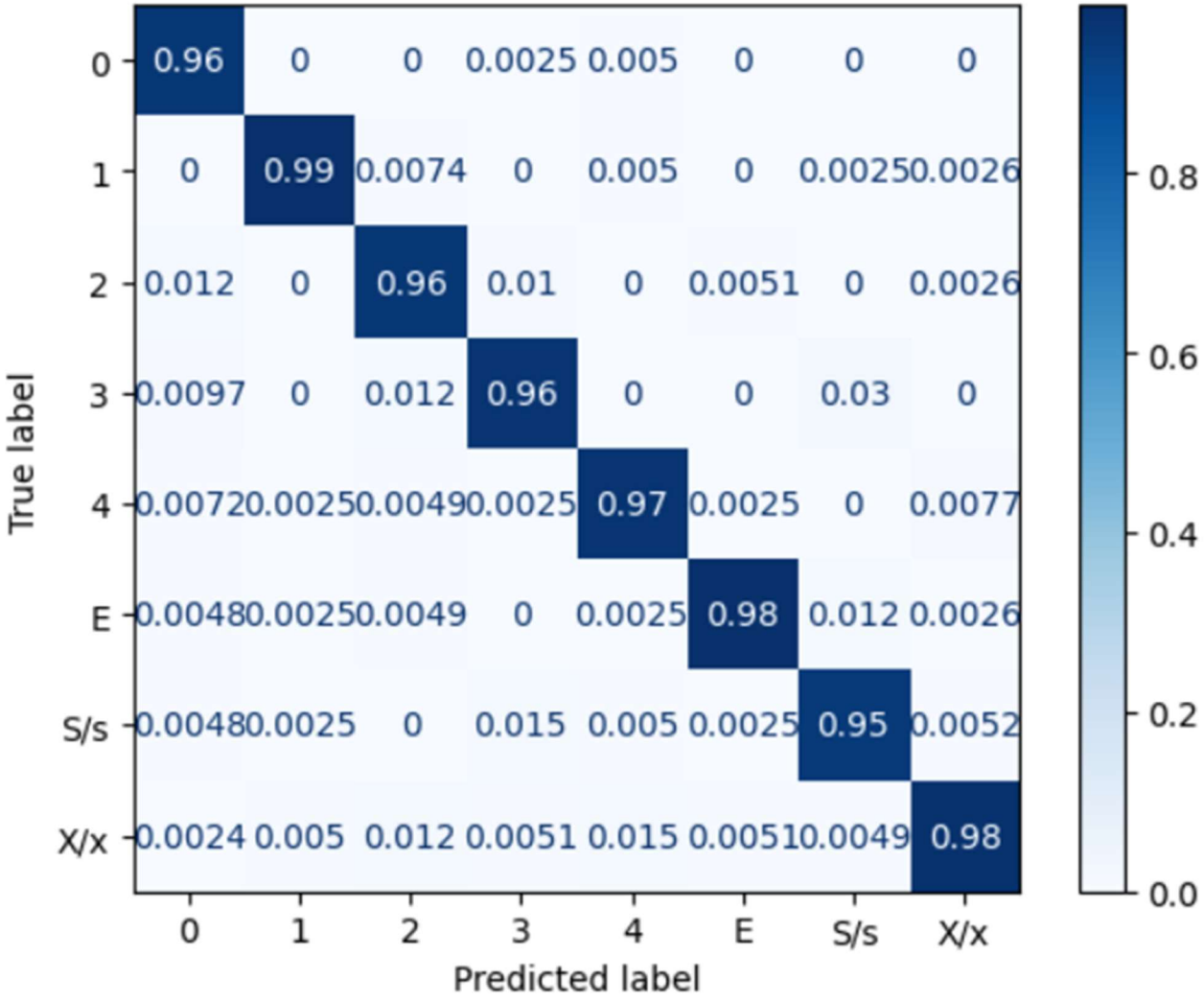
3.2.3 Loss Curves:



3.2.4 Accuracy Curves:



3.2.5 Confusion Matrix:



4. Algoritmi in fase di risoluzione

Per la risoluzione del problema sono stati utilizzati tre algoritmi diversi:

- Ricerca non informata: Breadth First Search (BFS)
- Ricerca informata: Greedy Best First, A*

4.1 Prestazioni degli algoritmi

Per testare le prestazioni degli algoritmi sono state utilizzate matrici quadrate di diverse dimensioni, in particolare tre ciascuna.

Per ogni matrice sono stati eseguiti gli algoritmi uno alla volta per calcolare il tempo di esecuzione in secondi e, per tutte le matrici di una certa dimensione è stata poi calcolata la media del tempo impiegato:

	4x4	**5x5**	**6x6**
BFS	0.00013446807861328125	0.00024620691935221356	0.00016864140828450522
Greedy	0.0002056757609049479	0.00023762385050455728	0.00041961669921875
A*	0.00019947687784830728	0.0002797444661458333	0.0003437201182047526

	7x7	**8x8**
BFS	0.0006751219431559244	0.0003921190897623698
Greedy	0.0009547869364420573	0.0007736682891845703
A*	0.0019495487213134766	0.0008389155069986979

4.2 Confronti fra algoritmi

Per realizzare i confronti fra i vari algoritmi sono state selezionate 3 matrici: una di dimensioni 5x5, una 7x7 e un'altra 8x8.

4.2.1 Matrice 5x5

4	3	3	1	1
S	4	4	3	2
0	X	1	1	0
0	X	3	X	E
1	1	2	X	X

```
p1 = EasyMaze(Labyrinth([
    ( 4 , 3 , 3 , 1 , 1 ),
    ('S', 4 , 4 , 3 , 2 ),
    ( 0 ,'X', 1 , 1 , 0 ),
    ( 0 ,'X', 3 ,'X','E'),
    ( 1 , 1 , 2 ,'X','X')
]))

sol_bfs = breadth_search_tree(p1)
show_solution("BFS: ", sol_bfs)
sol_greedy = greedy_best_first_search(p1)
show_solution("Greedy: ", sol_greedy)
sol_astar = astar_search(p1)
show_solution("A*: ", sol_astar)
```

```
BFS:  ['RIGHT', 'RIGHT', 'RIGHT', 'RIGHT', 'DOWN', 'DOWN']
Cost =  13
Path Length =  6
Greedy:  ['DOWN', 'DOWN', 'DOWN', 'RIGHT', 'RIGHT', 'UP', 'UP', 'RIGHT', 'RIGHT', 'DOWN']
Cost =  9
Path Length =  10
A*:  ['DOWN', 'DOWN', 'DOWN', 'RIGHT', 'RIGHT', 'UP', 'UP', 'RIGHT', 'RIGHT', 'DOWN']
Cost =  9
Path Length =  10
```

4.2.2 Matrice 7x7

0	E	0	X	E	X	X
1	2	0	0	2	X	X
0	2	X	X	3	X	X
1	0	X	X	3	X	4
X	0	1	X	4	4	S
X	X	1	2	0	0	X
X	X	X	X	2	3	X

```
p1 = EasyMaze(Labyrinth([
    ( 0 ,'E', 0 ,'X','E','X','X'),
    ( 1 , 2 , 0 , 0 , 2 ,'X','X'),
    ( 0 , 2 ,'X','X', 3 ,'X','X'),
    ( 1 , 0 ,'X','X', 3 ,'X', 4 ),
    ('X', 0 , 1 ,'X', 4 , 4 ,'S'),
    ('X','X', 1 , 2 , 0 , 0 ,'X'),
    ('X','X','X','X', 2 , 3 ,'X')
]))

sol_bfs = breadth_search_tree(p1)
show_solution("BFS: ", sol_bfs)
sol_greedy = greedy_best_first_search(p1)
show_solution("Greedy: ", sol_greedy)
sol_astar = astar_search(p1)
show_solution("A*: ", sol_astar)
```

```
BFS:  ['LEFT', 'LEFT', 'UP', 'UP', 'UP', 'UP']
Cost =  16
Path Length =  6
Greedy:  ['LEFT', 'DOWN', 'LEFT', 'LEFT', 'LEFT', 'UP', 'LEFT', 'UP', 'UP', 'LEFT', 'UP', 'UP', 'RIGHT']
Cost =  11
Path Length =  13
A*:  ['LEFT', 'DOWN', 'LEFT', 'LEFT', 'LEFT', 'UP', 'LEFT', 'UP', 'LEFT', 'UP', 'UP', 'UP', 'RIGHT']
Cost =  10
Path Length =  13
```

4.2.3 Matrice 8x8

X	X	4	S	4	X	X	X
X	X	3	2	4	2	X	X
1	2	3	2	1	2	0	X
E	2	X	0	2	1	0	X
0	0	1	1	0	X	X	X
2	X	X	1	3	2	X	X
2	1	X	1	X	1	0	3
X	0	E	0	X	X	0	E

```
p1 = EasyMaze(Labyrinth([
    ('X','X', 4 ,'S', 4 ,'X','X','X'),
    ('X','X', 3 , 2 , 4 , 2 ,'X','X'),
    ( 1 , 2 , 3 , 2 , 1 , 2 , 0 ,'X'),
    ('E', 2 ,'X', 0 , 2 , 1 , 0 ,'X'),
    ( 0 , 0 , 1 , 1 , 0 ,'X','X','X'),
    ( 2 ,'X','X', 1 , 3 , 2 ,'X','X'),
    ( 2 , 1 ,'X', 1 ,'X', 1 , 0 , 3 ),
    ('X', 0 ,'E', 0 ,'X','X', 0 ,'E')
]))

sol_bfs = breadth_search_tree(p1)
show_solution("BFS: ", sol_bfs)
sol_greedy = greedy_best_first_search(p1)
show_solution("Greedy: ", sol_greedy)
sol_astar = astar_search(p1)
show_solution("A*: ", sol_astar)
```

```
BFS:  ['DOWN', 'DOWN', 'LEFT', 'LEFT', 'DOWN', 'LEFT']
Cost =  11
Path Length =  6
Greedy:  ['DOWN', 'DOWN', 'DOWN', 'DOWN', 'LEFT', 'LEFT', 'LEFT', 'UP']
Cost =  6
Path Length =  8
A*:  ['DOWN', 'DOWN', 'DOWN', 'DOWN', 'LEFT', 'LEFT', 'LEFT', 'UP']
Cost =  6
Path Length =  8
```